



Green University of Bangladesh
Department of Computer Science and Engineering
(CSE)

Faculty of Sciences and Engineering
Semester: (Spring, Year: 2023), B.Sc. in CSE (Day)

LAB REPORT NO :03

Course Title: Artificial Intelligence Lab

Course Code: CSE-316

Section: 211D2

Lab Experiment Name: Iterative Deepening Depth First Search (IDDFS)

Student Details

Name		ID
1.	Md Mahdi Hasan	211002069

Lab Date : 19-03-24

Submission Date : 31-03-24

Course Teacher's Name : Md Naimul Pathan

Lab Report Status

Marks:

Comments:.....

Signature:.....

Date:.....

1. **TITLE:** Iterative Deepening Depth First Search (IDDFS).

2. **OBJECTIVES:** There are two common ways to traverse a graph, BFS and DFS. Considering a Tree (or Graph) of huge height and width, both BFS and DFS are not very efficient due to following reasons.

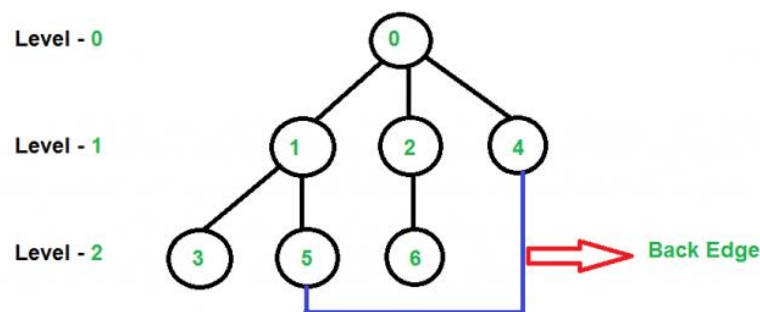
DFS: first traverses nodes going through one adjacent of root, then next adjacent. The problem with this approach is, if there is a node close to root, but not in first few subtrees explored by DFS, then DFS reaches that node very late. Also, DFS may not find shortest path to a node

BFS: goes level by level, but requires more space. The space required by DFS is $O(d)$ where d is depth of tree, but space required by BFS is $O(n)$ where n is number of nodes in tree.

IDDFS: combines depth-first search's space-efficiency and breadth-first searches.

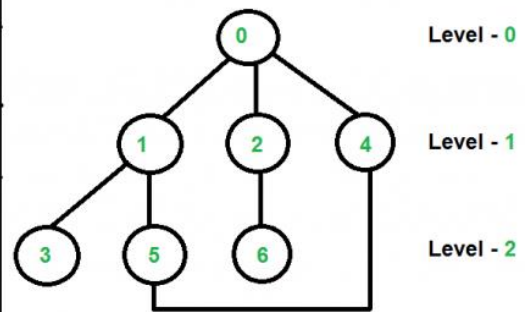
3. **PROCEDURE:** There can be two cases:

- When the graph has no cycle: This case is simple. We can DFS multiple times with different height limits.
- When the graph has cycles. This is interesting as there is no visited flag in IDDFS.



Although, at first sight, it may seem that since there are only 3 levels, so we might think that Iterative Deepening Depth First Search of level 3, 4, 5,...and so on will remain same. But, this is not the case. You can see that there is a cycle in the above graph, hence IDDFS will change for level-3,4,5..and so on.

Depth	Iterative Deepening Depth First Search
0	0
1	0 1 2 4
2	0 1 3 5 2 6 4 5
3	0 1 3 5 4 2 6 4 5 1



The explanation of the above pattern is left to the readers.

4. IMPLEMENTATION:

```

from collections import defaultdict

class Graph:

    def __init__(self, vertices):
        self.V = vertices
        self.graph = defaultdict(list)

    def addEdge(self, u, v):
        self.graph[u].append(v)

    def DLS(self, src, target, maxDepth):
        if src == target:
            return True
        if maxDepth <= 0:
            return False
        for i in self.graph[src]:
            print(i, end=' ')
            if self.DLS(i, target, maxDepth - 1):
                return True
        return False

    def IDDFS(self, src, target, maxDepth):
        print("Depth 0:", src, end=' ')
        for i in range(1, maxDepth + 1):
            print(f"\nDepth {i}: A ", end='')
            if self.DLS(src, target, i):
                return True
        return False

```

```

g = Graph(10)
g.addEdge('A', 'B')
g.addEdge('A', 'C')
g.addEdge('B', 'D')
g.addEdge('B', 'E')
g.addEdge('C', 'F')
g.addEdge('C', 'G')
g.addEdge('D', 'H')
g.addEdge('D', 'I')
g.addEdge('F', 'K')

target = 'G'
maxDepth = 3
src = 'A'

if g.IDDFS(src, target, maxDepth):
    print("\nTarget is reachable from source within max depth")
else:
    print("\nTarget is NOT reachable from source within max depth")

```

5. OUTPUT:

```

Depth 0: A
Depth 1: A B C
Depth 2: A B D E C F G
Target is reachable from source within max depth

```

6. ANALYSIS AND DISCUSSION: Iterative Deepening Depth First Search (IDDFS) efficiently explores large search spaces with limited memory by incrementally deepening the search limit. It ensures optimality in finding the shortest path while maintaining a low memory footprint similar to DFS. However, it may struggle with high branching factors and non-uniform edge costs.